SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

Total Marks: 30

QUESTIONS AND ANSWERS

1. Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

Question: 1. Identify appropriate HTML5 semantic elements missing in the structure.

Answer: Suitable semantic elements include <header> for the page heading, <main> for the main page content, and <section> for the registration area. A <footer> may be added for page information. The form should also have a descriptive heading and explicit labels for its inputs.

Question: 2. Modify the structure to make it standards-compliant.

Answer: Corrected HTML implementation:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>Online Symposium Registration</h1>
  </header>
  <main>
    <section aria-labelledby="registration-heading">
      <h2 id="registration-heading">Register for the Symposium</h2>
      <form action="/register" method="post">
        <label for="name">Full name:</label>
        <input type="text" id="name" name="name" required>

        <label for="email">Email address:</label>
        <input type="email" id="email" name="email" required>

        <button type="submit">Submit</button>
      </form>
    </section>
  </main>
  <footer>
    <p>Online Symposium Portal</p>
  </footer>
</body>
</html>
```

Question: 3. Interpret the HTTP request generated during form submission.

Answer: When the student submits the form, the browser sends a POST request to the /register endpoint. The named form values, such as name and email, are sent in the request body using the form's default URL-encoded format. The browser also sends headers, including the content type. Only successful controls with a name attribute are submitted. POST is suitable for sending registration data because the values are not placed in the URL query string.

Question: 4. Analyze the HTTP response cycle from server to browser.

Answer: The server receives the POST request, validates the submitted values, and processes the registration. It then returns an HTTP response containing a status code, response headers, and usually a response body. For example, 201 Created may indicate successful creation, 400 Bad Request may indicate invalid input, or 500 Internal Server Error may indicate a server-side problem. The browser interprets the status and displays the returned page or message. A redirect, such as 303 See Other, may be used after successful submission to prevent accidental resubmission on refresh.

Question: 5. Suggest improvements for usability and accessibility.

Answer: Add visible labels associated with each input, mark required fields with the required attribute, and use the correct input types. Provide clear instructions and helpful validation messages. Ensure keyboard navigation, visible focus indicators, sufficient colour contrast, and a meaningful page title. Validate all data on the server as well as in the browser, and use HTTPS to protect data in transit.

2. Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Question: 1. Identify improper nesting and missing closing tags.

Answer: The <h1> and <p> elements are missing their closing tags. The document also lacks the HTML5 doctype, a language attribute on the root <html> element, character encoding, and a viewport declaration. Although browsers often repair malformed markup automatically, relying on that recovery can cause inconsistent results.

Question: 2. Analyze how different browsers may interpret this structure.

Answer: Browsers use HTML parsing rules to construct a DOM and may automatically close or rearrange certain elements when tags are omitted. Their recovery can produce a page that appears different from the author's intention, especially when CSS is applied. Differences in default styles, form controls, fonts, and rendering engines can add further variation. Valid markup reduces ambiguity and makes rendering more predictable.

Question: 3. Identify missing semantic elements (e.g., header, section).

Answer: A <header> can contain the company heading, <main> can identify the primary content, and <section> can group the 'About us' content under a meaningful heading. A <footer> may be used for copyright or contact details. Semantic elements improve document structure, accessibility, and maintainability.

Question: 4. Provide a corrected W3C-compliant HTML structure.

Answer: Corrected HTML implementation:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
```

```
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section aria-labelledby="about-heading">
      <h2 id="about-heading">About Us</h2>
      <p>Learn more about our company and services.</p>
    </section>
  </main>
  <footer>
    <p>&copy; Company</p>
  </footer>
</body>
</html>
```

Question: 5. Suggest techniques to ensure cross-browser compatibility.

Answer: Use valid HTML and CSS, include a doctype, and validate the document with the W3C Markup Validation Service. Use a consistent CSS reset or normalize stylesheet where appropriate. Test in major browsers and at different viewport sizes, use feature detection instead of browser-specific assumptions, and avoid unsupported or non-standard features. Keep dependencies updated and check keyboard and screen-reader behaviour.

3. Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Question: 1. Identify issues in the form configuration.

Answer: The button has type="button", which does not submit the form by default. The form also uses GET for a password field. GET places form values in the URL query string, which is inappropriate for credentials because URLs may be stored in browser history, server logs, and other systems. The inputs would benefit from labels and required validation.

Question: 2. Analyze why the HTTP request is not triggered.

Answer: Clicking a button with type="button" does not trigger the browser's native form-submission behaviour. Unless JavaScript explicitly handles the click and sends a request, no form request is generated. Changing the button to type="submit" enables normal form submission.

Question: 3. Interpret the role of GET method in this scenario.

Answer: GET requests retrieve a resource and normally pass form values as URL query parameters. It is appropriate for non-sensitive searches or filters that should be bookmarkable, but it is not appropriate for sending passwords. GET does not provide confidentiality. HTTPS is needed to encrypt data in transit, and POST should be used for login credentials.

Question: 4. Propose a corrected implementation.

Answer: Corrected HTML implementation:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>Login</h1>
    <form action="/submit" method="post">
      <label for="username">Username:</label>
      <input type="text" id="username" name="username" autocomplete="username" required>

      <label for="password">Password:</label>
      <input type="password" id="password" name="password"
             autocomplete="current-password" required>
```

```
      <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

Question: 5. Suggest improvements for secure data transmission.

Answer: Serve the login page and endpoint only over HTTPS. Use POST for credentials, never store passwords in plain text, and verify passwords securely on the server using a modern password-hashing algorithm with a unique salt. Validate inputs on the server, use secure session cookies (Secure, HttpOnly, and an appropriate SameSite setting), implement rate limiting, and provide generic login error messages. Do not rely on client-side validation as a security control.

CODE OF CONDUCT

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____